```
# train.py (email.csv)
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import LabelEncoder
from sklearn.feature_extraction.text import CountVectorizer

from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier

import joblib
df = pd.read_csv("email.csv")

X = df["Message"]
y = df["Category"]

le = LabelEncoder()
y = le.fit_transform(y)

cv = CountVectorizer()
X = cv.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

models = {
    "KNN": KNeighborsClassifier(),
    "SVM": SVC(),
    "Decision Tree": DecisionTreeClassifier()
}

best_model = None
best_accuracy = 0
```

```
best_name = ""
print("Model Accuracy:\n")

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    print(f"{name}: {accuracy * 100:.2f}%")

    if accuracy > best_accuracy:
        best_model = model
        best_accuracy = accuracy
        best_name = name

print(f"\nBest Model: {best_name} ({best_accuracy * 100:.2f}%)")

joblib.dump(best_model, "best_model.pkl")
joblib.dump(cv, "vectorizer.pkl")
print("Model saved as best_model.pkl")
print("Vectorizer saved as vectorizer.pkl")

# test.py (email.csv)
import tkinter as tk
import joblib

try:
    model = joblib.load("best_model.pkl")
    vectorizer = joblib.load("vectorizer.pkl")
except:
    print("Run training file first and save model + vectorizer")
    exit()

def predict():
    message = entry.get()
```

```
    if message == "":
        result.config(text="Please enter a message")
        return

    message_vector = vectorizer.transform([message])

    prediction = model.predict(message_vector)[0]

    if prediction == 1:
        output = "Spam"
    else:
        output = "Not Spam"

    result.config(text=f"Prediction: {output}")

root = tk.Tk()
root.title("Spam Detection App")
root.geometry("400x300")

tk.Label(root, text="Enter Email Message", font=("Arial", 14)).pack(pady=10)

entry = tk.Entry(root, width=40)
entry.pack(pady=10)

tk.Button(root, text="Predict", command=predict).pack(pady=10)

result = tk.Label(root, text="", font=("Arial", 12))
result.pack(pady=10)

root.mainloop()
```

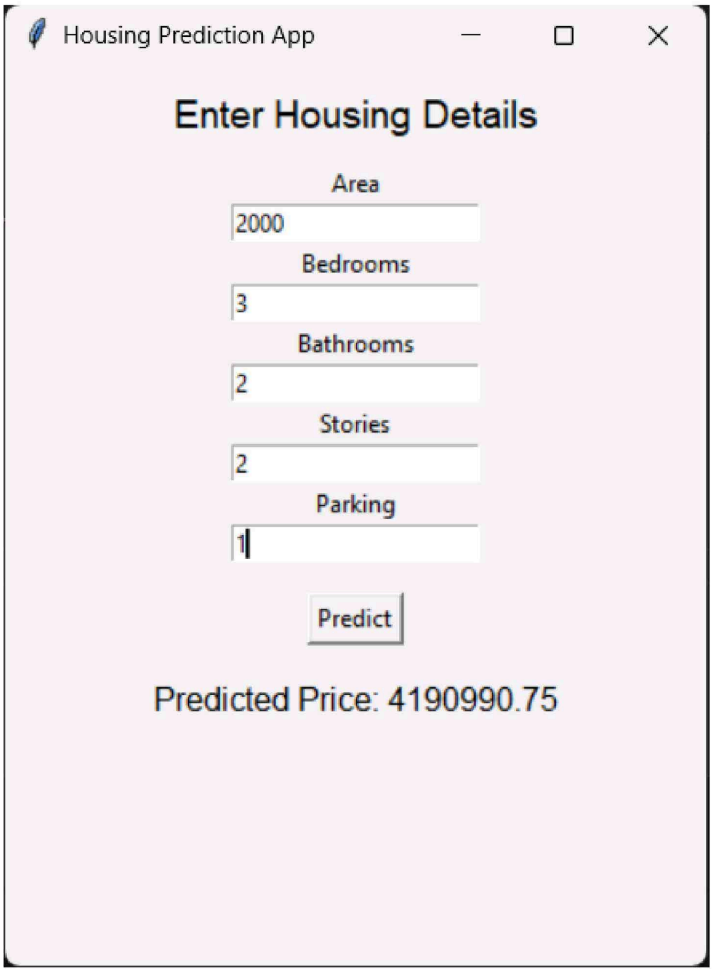
```
# train.py (housing.csv)
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
import joblib

df = pd.read_csv("housing.csv")
df = df.dropna()

df = pd.get_dummies(df, drop_first=True)
X = df.drop(columns=["price"])
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

models = {
    "Linear Regression": LinearRegression(),
    "Decision Tree": DecisionTreeRegressor(),
    "Random Forest": RandomForestRegressor()
}

best_model = None
best_score = 0
best_name = ""

print("Model Performance (R2 Score)")

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
```

```
    score = r2_score(y_test, y_pred)
    print(f"{name}: {score * 100:.2f}%")

    if score >= best_score:
        best_model = model
        best_score = score
        best_name = name

print(f"\nBest Model: {best_name} ({best_score * 100:.2f}%)")

joblib.dump(best_model, "housing_model.pkl")
print("Model saved as housing_model.pkl")

# test.py (housing.csv)
import tkinter as tk
import joblib
import pandas as pd

try:
    model = joblib.load("housing_model.pkl")
except:
    print("Please run htrain.py first")
    exit()

def predict():
    try:
        values = [
            float(entry1.get()),
            float(entry2.get()),
            float(entry3.get()),
            float(entry4.get()),
            float(entry5.get())
        ]

        df = pd.read_csv("housing.csv")
```

```
        df = df.dropna()
        df = pd.get_dummies(df, drop_first=True)

        columns = df.drop(columns=["price"]).columns
        data = pd.DataFrame([[0]*len(columns)], columns=columns)

        base_cols = ["area","bedrooms","bathrooms","stories","parking"]
        for i, col in enumerate(base_cols):
            data[col] = values[i]

        prediction = model.predict(data)[0]

        result.config(text=f"Predicted Price: {prediction:.2f}")

    except Exception as e:
        result.config(text=f"Error: {str(e)}")

root = tk.Tk()
root.title("Housing Prediction App")
root.geometry("350x450")

tk.Label(root, text="Enter Housing Details", font=("Arial", 14)).pack(pady=10)

tk.Label(root, text="Area").pack()
entry1 = tk.Entry(root)
entry1.pack()

tk.Label(root, text="Bedrooms").pack()
entry2 = tk.Entry(root)
entry2.pack()

tk.Label(root, text="Bathrooms").pack()
entry3 = tk.Entry(root)
entry3.pack()
```

```
tk.Label(root, text="Stories").pack()
entry4 = tk.Entry(root)
entry4.pack()

tk.Label(root, text="Parking").pack()
entry5 = tk.Entry(root)
entry5.pack()

tk.Button(root, text="Predict", command=predict).pack(pady=15)

result = tk.Label(root, text="", font=("Arial", 12))
result.pack()

root.mainloop()
```

Outputs:

```
(.venv) PS D:\Internship\Day44> python emtrain.py
Model Accuracy:

KNN: 92.56%
SVM: 98.12%
Decision Tree: 97.22%

Best Model: SVM (98.12%)
Model saved as best_model.pkl
```

```
(.venv) PS D:\Internship\Day44> python htrain.py
Model Performance (R2 Score)
Linear Regression: 65.29%
Decision Tree: 42.44%
Random Forest: 61.51%

Best Model: Linear Regression (65.29%)
Model saved as housing_model.pkl
```